

8.4.1 Some simple examples

Let's revisit an example that we mentioned at the start of this chapter—specifying the behavior of the function `max`, available as a method on `List` (API docs link: <http://mng.bz/Pz86>). The maximum of a list should be greater than or equal to every other element in the list. Let's specify this:

```
val smallInt = Gen.choose(-10,10)
val maxProp = forAll(listOf(smallInt)) { ns =>
  val max = ns.max
  !ns.exists(_ > max)
}
```

No value greater than
`max` should exist in `ns`.

At this point, calling `run` directly on a `Prop` is rather cumbersome. We can introduce a helper function for running our `Prop` values and printing their result to the console in a useful format. Let's make this a method on the `Prop` companion object.

Listing 8.5 A run helper function for `Prop`

```
def run(p: Prop,
  maxSize: Int = 100,
  testCases: Int = 100,
  rng: RNG = RNG.Simple(System.currentTimeMillis)): Unit =
  p.run(maxSize, testCases, rng) match {
    case Falsified(msg, n) =>
      println(s"! Falsified after $n passed tests:\n $msg")
    case Passed =>
      println(s"+ OK, passed $testCases tests.")
  }
```

A default argument of 100

We're taking advantage of default arguments here. This makes the method more convenient to call. We want the default number of tests to be enough to get good coverage, but not too many or they'll take too long to run.

If we try running `run(maxProp)`, we notice that the property fails! Property-based testing has a way of revealing hidden assumptions that we have about our code, and forcing us to be more explicit about these assumptions. The standard library's implementation of `max` crashes when given the empty list. We need to fix our property to take this into account.



EXERCISE 8.13

Define `listOf1` for generating nonempty lists, and then update your specification of `max` to use this generator.

Let's try a few more examples.